ESCOLA NAVAL

MARINHA PORTUGUESA

INTELIGÊNCIA ARTIFICIAL

Ano letivo 2026–2027



Docente: António Gonçalves

Como encontra o robô um caminho até ao objetivo?

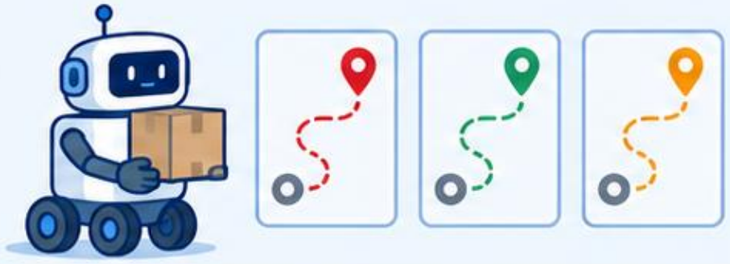
Na primeira semana vimos como o agente decide. Agora vamos perceber como constrói um plano antes de agir.

1 O que já sabemos

O robô tem uma tarefa, observa o ambiente e escolhe ações para atingir um objetivo.

Definimos requisitos e critérios para comparar percursos. Na arquitetura híbrida, o planeamento propõe caminhos até ao destino.

Nos exemplos anteriores, os percursos candidatos já eram apresentados.



2 A pergunta que falta

Como encontra o programa os percursos possíveis?

Conhecer o destino não indica que movimentos devem ser executados.

É necessário usar o mapa e as ações permitidas para construir uma sequência que leve o robô do início ao destino.



3 O que vamos estudar

Representar a situação inicial, as ações e os seus efeitos.

Explorar alternativas com diferentes estratégias de procura.

Verificar se o plano cumpre os requisitos e comparar o seu custo.

A solução procurada é uma sequência de ações.

O mesmo robô, uma nova pergunta



O robô parte de **B** com 8 unidades de energia e deve chegar a **D** com uma reserva mínima de 2 unidades.

Que sequência de movimentos permite cumprir a tarefa?

Para começar: mapa e custos conhecidos, movimentos com efeitos previsíveis e ambiente estável durante o planeamento.

A procura permite encontrar planos para um agente orientado para objetivos.

Formulação e procura não informada

O que vamos aprender nesta aula

1. **Formular o problema**
   Estado inicial, ações, transições, objetivo, restrições e custos.

2. **Representar a procura**
   Estados, nós e alternativas por explorar.

3. **Aplicar três estratégias**
   Procura em largura (BFS), em profundidade (DFS) e de custo uniforme (UCS).

4. **Comparar os resultados**
   Caminho encontrado, custo da solução, tempo e memória da procura.

5. **Verificar a solução e as garantias**
   Cumprimento dos requisitos e condições para encontrar uma solução ou uma solução de menor custo.

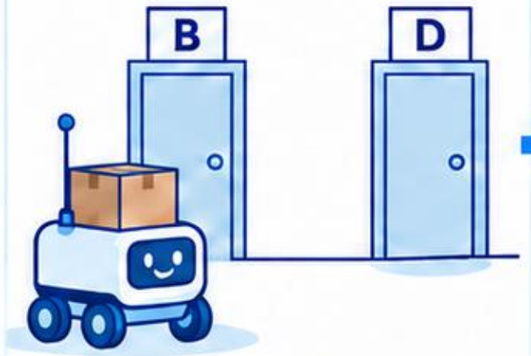
Começamos por transformar a tarefa do robô em requisitos concretos.

Do PEAS aos requisitos do problema

Na primeira semana usámos o PEAS para descrever a tarefa do robô. Agora usamos essa informação para preparar a procura de um plano.

Tarefa e informação inicial

Transportar uma encomenda de B para D.



Neste exemplo, o robô começa em B com **12 unidades** de energia.

S — Sensores

Fornecem informação para estimar a posição e medir a carga disponível.

Condições obrigatórias

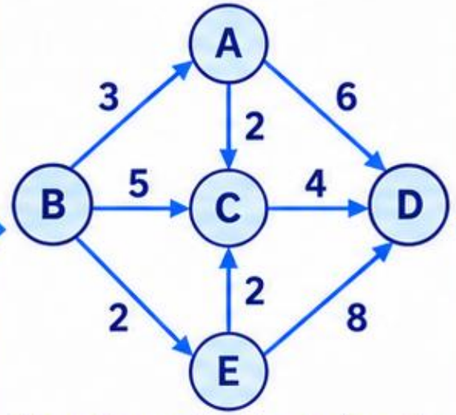
P — Medida de desempenho

- Chegar ao destino D.
- Usar apenas ligações permitidas.
- Ter energia para cada movimento.
- Terminar com uma reserva mínima de 2 unidades.

Um caminho válido cumpre todas estas condições.

Mapa e movimentos

E — Ambiente



Mapa fornecido: locais, ligações permitidas e energia consumida.

A — Atuadores

Os motores executam os movimentos escolhidos.

u.e. = unidade de energia (dados didáticos).

Critério de escolha

P — Medida de desempenho

Como comparar os caminhos válidos?

Entre os caminhos que cumprem todas as condições, queremos encontrar o que **consome menos energia.**



Minimizar a energia consumida.

O que verificar?

- O caminho chega a D?
- Todos os movimentos são permitidos e executáveis?
- A energia final é suficiente?
- Existe outro caminho válido que consuma menos?

No PEAS, **A** significa **atuadores.** Na procura, as **ações** são os movimentos que esses atuadores executam.

O PEAS descreve a tarefa do agente.

A formulação da procura define os estados e as ações para encontrar um plano.

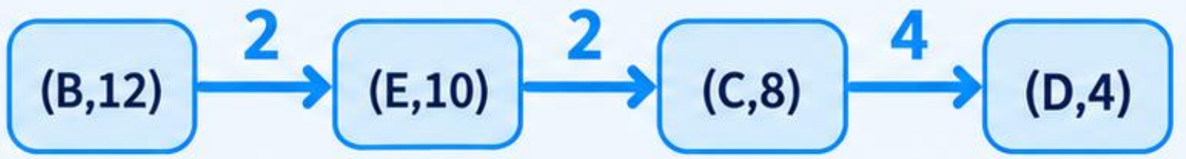
A seguir: estado inicial, ações, transições, objetivo e custos.

Como formular um problema de procura

Da tarefa do robô à descrição usada pelo algoritmo

$$P = (S, A, T, s_0, G, c)$$

Os seis componentes	Exemplo do robô
S	Estados: posição e energia, por exemplo (B,12)
$A(s)$	Ações: ligações permitidas com energia suficiente
$T(s,a)$	Transição: de (B,12) para (E,10)
s_0	Estado inicial: (B,12)
$G(s)$	Objetivo: estar em D com energia ≥ 2
$c(s,a,s')$	Custo da ação: B para E consome 2 unidades



Custo total: 2 + 2 + 4 = 8 unidades

Solução válida: chega a D com 4 unidades de energia.

Uma solução ótima é uma solução válida com menor custo. A validade, por si só, não garante que seja ótima.



Nesta aula: modelo conhecido, transições determinísticas, estados discretos e ambiente estático.
A formulação define o que o algoritmo pode fazer e como reconhecer uma solução.

Vamos pensar

Aplicar os conceitos a uma nova situação

Analise o problema, formule uma resposta e justifique o seu raciocínio.



Objetivo e solução ótima

QUESTÃO

Um robô começa em B com 12 unidades de energia. Deve chegar a D com pelo menos 2 unidades.
O estado é (posição, energia). Cada movimento consome energia; o custo total é a soma dos consumos.



Grupo A:

«Estar em D com energia suficiente.»



Grupo B:

«Estar em D, com pelo menos 2 unidades, tendo percorrido o caminho de menor custo.»



1 **Reformule a proposta do grupo A como uma condição precisa, que possa ser testada usando apenas o estado atual.**

2 **A proposta do grupo B mistura duas verificações. Identifique-as e escreva separadamente o objetivo e o critério de otimização.**

3 **O robô chega a (D, 3) por um percurso com todas as ações permitidas.**
A solução é válida? É necessariamente ótima? Justifique ambas as respostas.

Objetivo e solução ótima

QUESTÃO

Um robô começa em B com 12 unidades de energia. Deve chegar a D com pelo menos 2 unidades. O estado é (posição, energia). Cada movimento consome energia; o custo total é a soma dos consumos.

Grupo A:
«Estar em D com energia suficiente.»

Grupo B:
«Estar em D, com pelo menos 2 unidades, tendo percorrido o caminho de menor custo.»

1. **Reformule a proposta do grupo A como uma condição precisa, que possa ser testada usando apenas o estado atual.**

2. **A proposta do grupo B mistura duas verificações. Identifique-as e escreva separadamente o objetivo e o critério de otimização.**

3. **O robô chega a (D, 3) por um percurso com todas as ações permitidas. A solução é válida? É necessariamente ótima? Justifique ambas as respostas.**

SOLUÇÃO

1 Condição de objetivo

posição = D E energia ≥ 2

Substituir «energia suficiente» por uma reserva mínima de 2 unidades. O teste usa a posição e a energia do estado atual.

2 Objetivo e critério de otimização

O grupo B mistura verificar se o objetivo foi atingido com determinar se o caminho é o mais barato.

Objetivo: chegar a D com energia ≥ 2.

Critério: minimizar o custo total do percurso.

3 Validade e optimalidade em (D, 3)

É válida: chega a D com 3 ≥ 2 unidades e todas as ações foram permitidas.

Não é necessariamente ótima: este percurso custou 12 − 3 = 9 unidades, mas pode existir outro percurso válido com custo inferior.

Para provar optimalidade, é necessário comparar alternativas ou usar uma estratégia com essa garantia.

Uma solução válida cumpre os requisitos.
Uma solução ótima tem o menor custo entre as válidas.

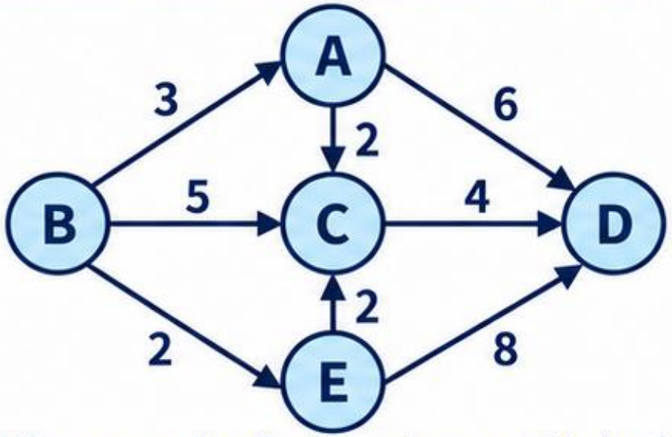
Estado, ação e transição

O grafo mostra as ligações; o estado indica a posição e a energia do robô.

Vamos aplicar o modelo: representar a situação do robô e calcular o que muda quando se desloca.

1. Grafo e estado do robô

O robô só pode deslocar-se pelas ligações indicadas. Cada movimento consome energia.



Números nas ligações: energia consumida (u.e.).

Estado = (posição, energia)

$$s = (p, e)$$

Estado inicial: (B, 12)

O robô está em B e dispõe de 12 unidades de energia.

Os pontos do grafo representam locais.
O estado inclui também a energia.

2. Ação e transição

A ação $B \to C$ muda a posição e reduz a energia disponível.



(B, 12)
Antes da ação

(C, 7)
Depois da ação

Energia final: $12 - 5 = 7$

A ação é permitida porque:

- Existe a ligação $B \to C$.
- A energia disponível é suficiente: $12 \geq 5$.

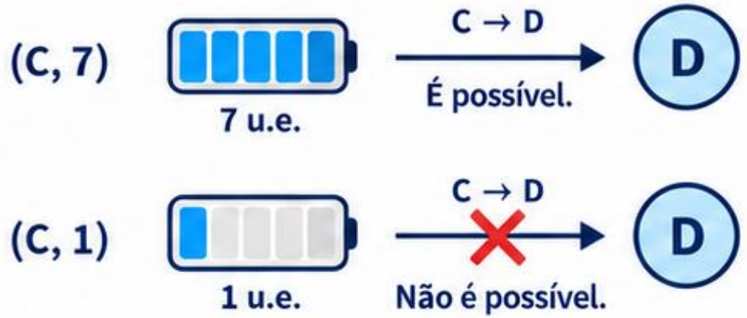
A transição calcula o estado resultante da ação.

$$T((B, 12), B \to C) = (C, 7)$$

3. A posição não chega

No mesmo local, energias diferentes podem permitir ações diferentes.

Estado	Pode executar C → D?
$(C, 7)$	**Sim:** tem 7 e precisa de 4.
$(C, 1)$	**Não:** tem 1 e precisa de 4.



A energia faz parte do estado porque influencia os movimentos que o robô pode executar.

$(C, 1)$ é uma situação hipotética para comparação; não resulta da transição mostrada.

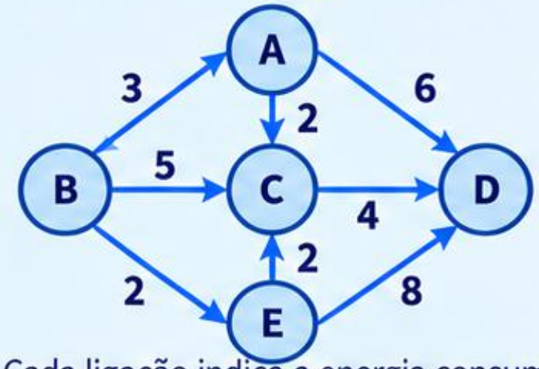
O grafo indica as ligações possíveis. A posição e a energia determinam quais as ações que o robô pode executar.

Objetivo, restrições e custo

Cumprir os requisitos e escolher a solução de menor custo

Já sabemos calcular o resultado de uma ação. Agora vamos verificar se um percurso resolve o problema e comparar o seu custo.

1. O mesmo problema





Cada ligação indica a energia consumida (u.e.).

Estado inicial: (*B*, 12)
O robô parte de B com 12 unidades de energia.

Só pode usar ligações existentes e executar movimentos para os quais tenha energia suficiente.

2. Quando cumpre o objetivo?

A entrega está concluída quando o robô chega a D e conserva, pelo menos, 2 unidades de energia.

Posição final = D **E** **Energia final ≥ 2**

$$G(p, e) = (p = D) \wedge (e \geq 2)$$

As duas condições têm de ser cumpridas.

Um estado intermédio pode ser válido sem satisfazer o objetivo.

3. Como comparamos os percursos?

O custo de um percurso é a soma da energia consumida em todas as suas ações.

Custo total = soma dos custos das ações

Energia final = 12 – custo total

Entre os percursos que cumprem os requisitos, procuramos o que consome menos energia.

Importante:
Neste exemplo, o custo é medido em unidades de energia (u.e.).

Dois percursos admissíveis, custos diferentes

Percurso	N.º de ações	Custo total (u.e.)	Energia final (u.e.)	Admissível?	Comparação
B → C → D	2	5 + 4 = 9	12 – 9 = 3	Sim	Cumpre os requisitos
B → E → C → D	**3**	**2 + 2 + 4 = 8**	**12 – 8 = 4**	**Sim**	**Menor custo: solução ótima**

Escolhemos **B → E → C → D** porque consome menos energia (8 < 9) e mantém a reserva exigida.

Uma solução admissível cumpre os requisitos. Uma solução ótima tem o menor custo entre todas as soluções admissíveis.

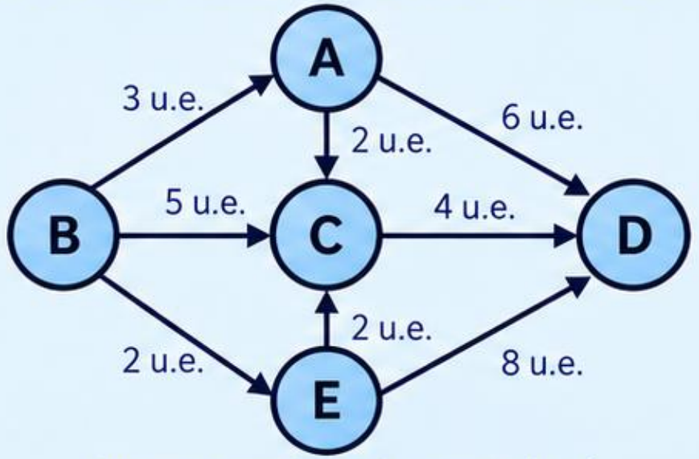
Do estado ao nó de procura

O estado descreve a situação; o nó guarda também como chegámos até ela.

Mantemos o mesmo grafo e o estado inicial (B, 12). Durante a procura, guardamos os estados alcançados e a informação sobre cada percurso.

1. O grafo de locais

O mapa indica os locais, as ligações permitidas e o consumo de energia de cada movimento.



Custos em unidades de energia (u.e.).

Estado inicial: (B, 12)
Posição B; energia disponível 12.

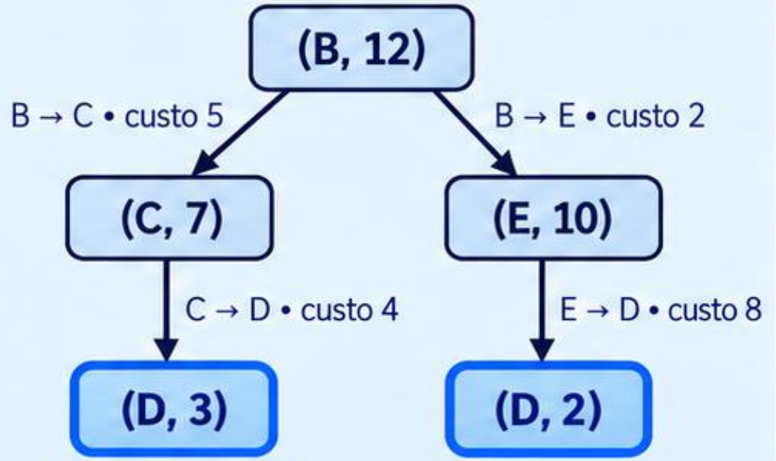
Estado = (posição, energia)
Exemplos: (C, 7), (D, 3) e (D, 2).

Os pontos deste mapa são locais; o estado do robô inclui também a energia.

2. A árvore de procura

Excerto da árvore: dois percursos.

Cada nó representa um estado alcançado por um determinado caminho.



Exemplo detalhado no bloco 3

Mesmo local, estados diferentes
(D, 3) e (D, 2) têm a mesma posição, mas energias diferentes.

3. O que guarda um nó?

Exemplo: nó alcançado pelo percurso B → C → D.

Campo	Valor neste exemplo
Estado	(D, 3)
Pai	Nó com estado (C, 7)
Ação	C → D
Custo acumulado, *g*	5 + 4 = 9 u.e.
Profundidade, d	2 ações

O custo soma a energia consumida desde o início.
A profundidade conta as ações executadas.

nó = (estado, pai, ação, g, d)

Como recuperamos o percurso?

Seguimos os nós pai: (D, 3) → (C, 7) → (B, 12).
Invertendo a ordem, obtemos o percurso: B → C → D.
(Esta sequência é obtida seguindo os nós pai, não são movimentos no grafo.)

O estado descreve a situação do robô. O nó de procura guarda também o caminho que permitiu alcançá-la.

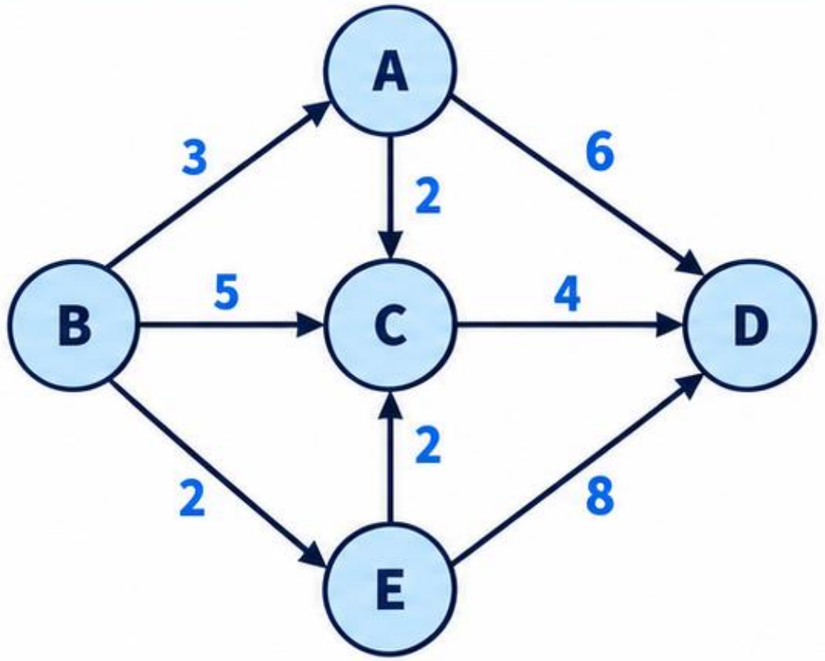
Fronteira e ciclo geral de procura

O mapa mostra as ligações. A fronteira guarda os nós que aguardam seleção.

1. Mapa do problema

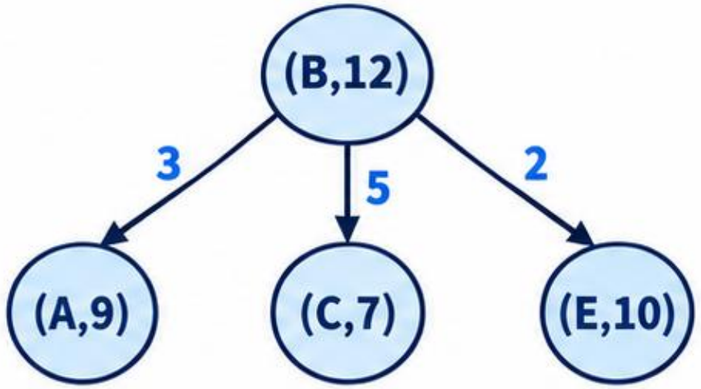
Estado inicial:
(B,12)

Objetivo:
D com energia ≥ 2



O mapa mostra as ligações disponíveis.

2. Evolução da fronteira



1	**Inicializar:**	[(B,12)]
2	**Retirar (B,12):**	[]
3	**Testar objetivo:**	[]
4	Expandir e inserir:	[(A,9), (C,7), (E,10)]

A fronteira fica temporariamente vazia após a retirada. O nó é processado antes da próxima verificação.

A ordem de seleção depende da estratégia.

3. Ciclo de procura

1. **Fronteira vazia?**
   Terminar sem solução.
2. **Retirar um nó da fronteira.**
3. **Objetivo atingido?**
   Devolver o caminho.
4. **Caso contrário, expandir e atualizar a fronteira.**
5. **Repetir.**

Gerar: criar um sucessor.
Retirar: selecionar um nó da fronteira.
Expandir: gerar sucessores válidos.

O grafo descreve o problema. A fronteira mostra os nós disponíveis para analisar a seguir.

Vamos pensar

Aplicar os conceitos a uma nova situação

Analise o problema, formule uma resposta e justifique o seu raciocínio.



Mesmo estado, mesmo nó?

QUESTÃO

Um robô começa em B com 12 unidades de energia.
O estado é (posição, energia). Cada ação consome a energia indicada; não há recarga.



Percurso 1: B → C



Percurso 2: B → A → C



O custo acumulado soma os consumos. A profundidade conta as ações.

Aluno A:
«Caminhos diferentes levam necessariamente a estados diferentes.»

Aluno B:
«Se o estado final é igual, os nós de procura são iguais em todos os campos.»

1. Calcule a energia final de cada percurso e indique se chegam ao mesmo estado.
2. Compare os dois nós em C: estado, pai, custo acumulado e profundidade.
3. Avalie as duas afirmações. Justifique por que um mesmo estado pode aparecer em nós de procura diferentes.

Mesmo estado, mesmo nó?

QUESTÃO

Um robô começa em B com 12 unidades de energia.
O estado é (posição, energia). Cada ação consome a energia indicada; não há recarga.



Percurso 1: B → C



Percurso 2: B → A → C



O custo acumulado soma os consumos. A profundidade conta as ações.

Aluno A:
«Caminhos diferentes levam necessariamente a estados diferentes.»

Aluno B:
«Se o estado final é igual, os nós de procura são iguais em todos os campos.»

1. Calcule a energia final de cada percurso e indique se chegam ao mesmo estado.
2. Compare os dois nós em C: estado, pai, custo acumulado e profundidade.
3. Avalie as duas afirmações. Justifique por que um mesmo estado pode aparecer em nós de procura diferentes.

SOLUÇÃO

1 Estado final

Percurso 1: 12 − 5 = 7
Percurso 2: 12 − 3 − 2 = 7

Ambos chegam ao estado (C, 7).

2 Informação guardada nos nós

Campo	B → C	B → A → C
Estado	(C, 7)	(C, 7)
Pai	Nó (B, 12)	Nó (A, 9)
Custo g	5	3 + 2 = 5
Profundidade d	1 ação	2 ações

3 Avaliação das afirmações

Aluno A: incorreto. Os percursos são diferentes, mas a posição e a energia finais são iguais.

Aluno B: incorreto. O estado é o custo coincidem, mas o pai e a profundidade diferem.



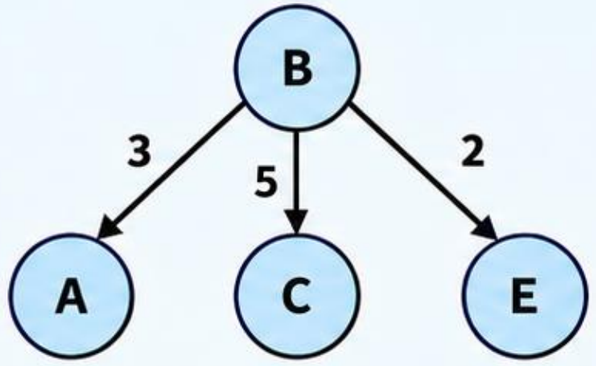
O estado descreve a situação.
O nó regista também como chegámos até ela.

Três regras para selecionar o próximo nó

Procura em largura (BFS), em profundidade (DFS) e de custo uniforme (UCS)

Mesmo mapa, modelo simplificado: para comparar as **estratégias**, o **estado** é apenas o local e mantemos os custos.
A reserva de energia será verificada no percurso encontrado: 12 – custo ≥ 2.

Ligações a partir de B



B é o estado inicial: g(B) = 0.

Depois de retirar e expandir B, inserimos os sucessores pela ordem A, C, E.

Sucessor	Custo da ação desde B
A	3
C	5
E	2

Os números nas ligações indicam o custo de cada ação. Como B é o início, g(B) = 0.

BFS — Largura

Primeiro a entrar, primeiro a sair

Usa uma fila e seleciona os nós por menor profundidade. Entre estes sucessores, retira o primeiro inserido.



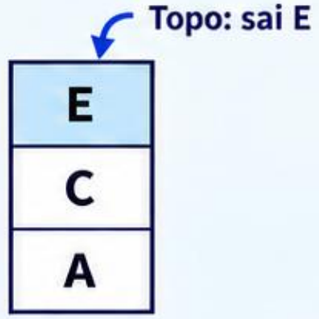
Próximo nó a retirar: A

A foi o primeiro a entrar na fila.

DFS — Profundidade

Último a entrar, primeiro a sair

Usa uma pilha. Com a inserção A, C, E, o nó E fica no topo.



Próximo nó a retirar: E

E foi o último a entrar na pilha.
A ordem de inserção influencia o ramo seguido.

UCS — Custo uniforme

Menor custo acumulado primeiro

Retira, de toda a fronteira, o nó com menor custo acumulado desde o início.

g(filho) = g(pai) + custo da ação

Nó	Custo acumulado g
E	0 + 2 = 2
A	0 + 3 = 3
C	0 + 5 = 5

Primeiro nó a retirar: E

No passo seguinte

Ao expandir E: C fica com g = 2 + 2 = 4 e D com g = 2 + 8 = 10.

A: 3	C: 4	D: 10

A UCS retira A, que já estava na fronteira.

Mesma escolha, razões diferentes

DFS e UCS escolhem **E** neste momento. A DFS considera a ordem de inserção; a UCS considera o custo acumulado.
O controlo de estados repetidos será explicado nos exemplos de cada algoritmo.

A estratégia determina a ordem de retirada da fronteira; não altera os custos das ligações.

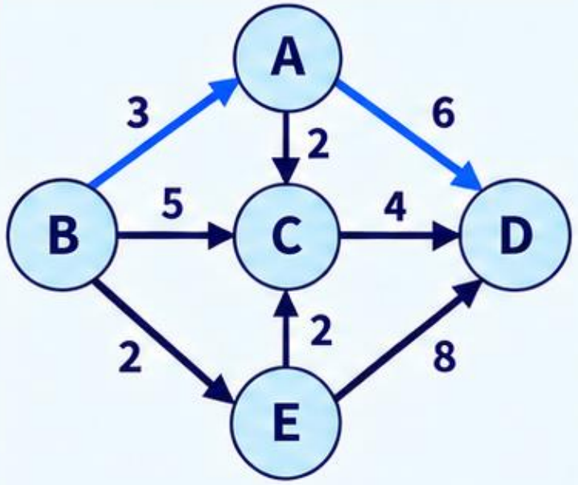
BFS: procura em largura

Explorar por níveis: primeiro 1 ação, depois 2 ações

1. Do grafo à árvore de descoberta

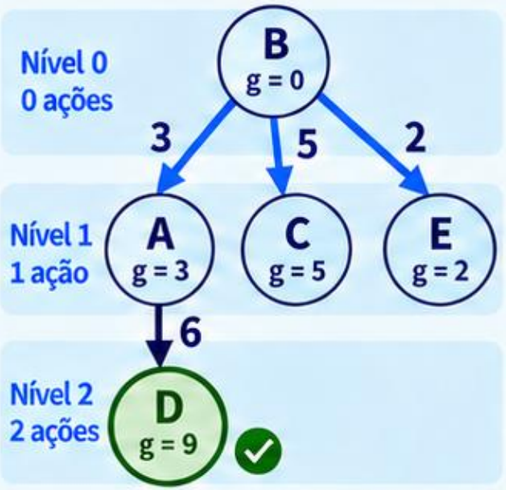
Partimos de **B** e procuramos D. O estado é apenas o local.

Grafo completo



Nas ligações: custo da ação.

Árvore de descoberta da BFS



Objetivo encontrado
ao retirar D da fila

A, C e E são descobertos ao expandir **B**. **D** é descoberto ao expandir **A**.
C e D não são reinseridos quando voltam a surgir.
A, C e E sãoretirados antes de **D**. Os níveis contam ações, não custos.

2. Fila e estados alcançados

Fila FIFO: primeiro a entrar, primeiro a sair.

Depois de expandir B

Fila



Alcançados: {B, A, C, E}

Retirar e expandir A

- **C** — já alcançado: não reinserir.
- **D** — novo: inserir e guardar pai(D) = A.

Fila



Alcançados: {B, A, C, E, D }

Retirar da fila não apaga o estado dos alcançados.
D guarda a retirada de C e E.

3. Algoritmo em linguagem simples

1. Criar o nó inicial com g = 0, colocá-lo na fila e marcar o seu estado como alcançado.
2. **Enquanto** a fila não estiver vazia:
   - Retirar o primeiro nó.
   - Se o seu estado satisfaz o objetivo, devolver o caminho.
   - Gerar sucessores por ordem alfabética.
   - Para cada sucessor cujo estado ainda não esteja nos alcançados:
     - Guardar o pai e a ação.
     - Calcular g(filho) = g(pai) + custo da ação.
     - Marcar o estado como alcançado.
     - Inserir o nó no fim da fila.
3. Se a fila se esgotar, devolver falha.

g regista o custo do caminho; não altera a ordem FIFO.
A fila guarda nós; os alcançados guardam estados.

O que garante?

- Se existir uma solução e a ramificação for finita, a BFS encontra uma solução com o menor número de ações.
- Com custos iguais e positivos, essa solução também tem o menor custo.

A BFS explora por níveis. A fila garante que os nós menos profundos são retirados primeiro.

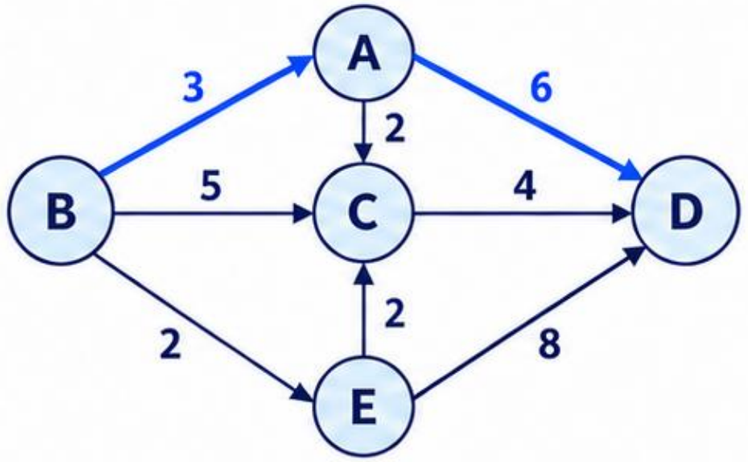
BFS: execução passo a passo

Seguir a fila e compreender o caminho encontrado

Início: B • Objetivo: D • Sucessores por ordem alfabética • Marcar ao inserir • Testar o objetivo ao retirar

1. O grafo completo

O mesmo exemplo: o estado é apenas o local.
Os números representam custos.



Nas setas: custo de cada ação.

O custo das ligações não determina a ordem da fila.
Guardamos o pai de cada estado quando o inserimos pela primeira vez.

2. Evolução da fila

O primeiro nó da fila está à esquerda.

Operação	Fila	d	Explicação
Inicializar	[B]	—	Inserir o nó inicial.
Retirar e expandir B	[A, C, E]	0	Inserir A, C e E.
Retirar e expandir A	[C, E, D]	1	C já foi alcançado. Inserir D, com pai A.
Retirar e expandir C	[E, D]	1	D já foi alcançado; não voltar a inserir.
Retirar e expandir E	[D]	1	C e D já foram alcançados.
Retirar D	[ ]	2	Objetivo encontrado. D não é expandido.

D entra na fila ao expandir A, mas a procura só termina quando D é retirado.

A, C e E são retirados antes de D porque estão a menor profundidade.

Concluímos o nível 1 antes de retirar D, no nível 2.

3. Resultado

Caminho encontrado
B → A → D

Custo total
3 + 6 = 9

Profundidade
2 ações

Ordem de retirada: B, A, C, E, D.

5 nós retirados

4 nós expandidos

Foram expandidos B, A, C e E.
D foi retirado e reconhecido como objetivo.

Menos ações não significa menor custo

B → **A** → D: 2 ações, custo 9 — caminho encontrado pela BFS.
B → **E** → **C** → D: 3 ações, custo 2 + 2 + 4 = 8 — mais barato, mas mais longo em número de ações.

A BFS dá prioridade ao número de ações, não ao custo acumulado.

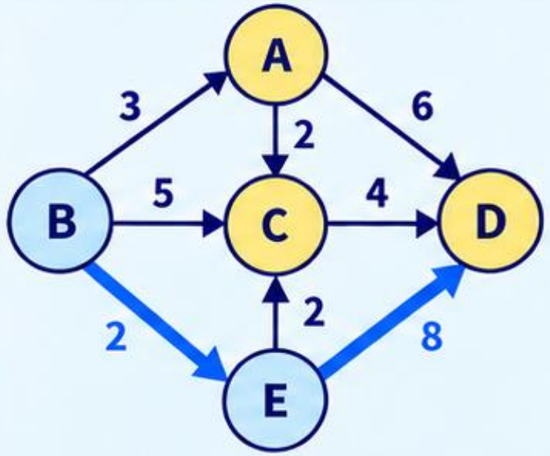
DFS: procura em profundidade

Aprofundar um ramo antes de explorar alternativas

1. Do grafo à árvore de descoberta

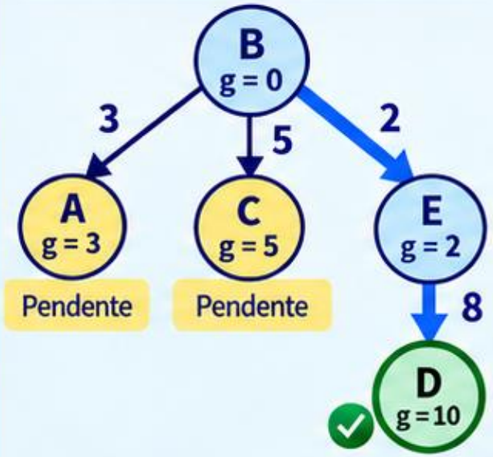
Partimos de B e procuramos D. O estado é apenas o local.

Grafo completo



Números nas ligações = custos das ações.

Árvore de descoberta da DFS



g = custo acumulado desde B.
A DFS não usa g para selecionar nós.

Objetivo encontrado ao retirar D da pilha

Com inserção A, C, E, retiramos E primeiro e seguimos B → E → D.
C já foi alcançado ao expandir B; não é reinserido ao expandir E.
A e C permanecem na pilha quando retiramos D e terminamos.

2. Pilha e estados alcançados

Pilha LIFO: último a entrar, primeiro a sair.

Depois de expandir B



Retirar / inserir no topo

Alcançados: {B, A, C, E}

Retirar e expandir E

C — já alcançado: não reinserir.

D — novo: guardar pai(D) = E.



Alcançados: {B, A, C, E, D}

Retirar da pilha não apaga o estado dos alcançados.
D sai a seguir; A e C ficam pendentes.

3. Algoritmo em linguagem simples

1. Criar o nó inicial com g = 0, colocá-lo na pilha e marcar o seu estado como alcançado.
2. Enquanto a pilha não estiver vazia:
   - Retirar o nó do topo.
   - Se o seu estado satisfaz o objetivo, devolver o caminho.
   - Gerar sucessores por ordem alfabética.
   - Para cada sucessor cujo estado ainda não esteja nos alcançados:
     - Guardar o pai e a ação.
     - Calcular g(filho) = g(pai) + custo da ação.
     - Marcar o estado como alcançado.
     - Inserir o nó no topo da pilha.
3. Se a pilha se esgotar, devolver falha.

g regista o custo do caminho; não altera a ordem LIFO.
A pilha guarda nós; os alcançados guardam estados.

O que garante?

- Num grafo finito, com controlo de estados repetidos, encontra uma solução se existir.
- Num espaço infinito, pode prosseguir indefinidamente num ramo.
- Não garante o menor número de ações nem o menor custo.

A DFS aprofunda um ramo. A pilha guarda as alternativas que poderão ser retomadas.

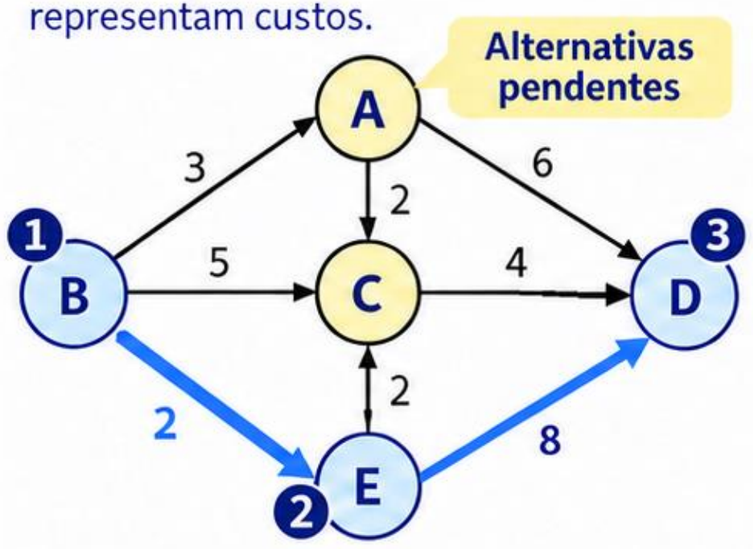
DFS: execução passo a passo

Aprofundar o ramo B → E → D e manter alternativas na pilha

Início: B • Objetivo: D • Inserção alfabética • Marcar ao inserir • Testar ao retirar

1. O mesmo grafo

Estado: apenas o local. Os números representam custos.



Os custos não determinam a ordem da pilha.

2. Evolução da pilha

Topo à direita: o último elemento é retirado primeiro.

Operação	Pilha	Explicação
Inicializar	[B]	Inserir B.
Retirar e expandir B	[A, C, E]	Inserir A, C e E. E fica no topo.
Retirar e expandir E	[A, C, D]	C já foi alcançado. Inserir D, com pai E.
Retirar D	[A, C]	Objetivo encontrado. D não é expandido.

Pilha no fim: alternativas não exploradas. [A] [C] ← Topo

- A DFS segue **B → E → D** antes de retomar **A** ou **C**. Como **D** é o objetivo, termina sem precisar de regressar.
- **C** já tinha sido alcançado ao expandir B; não é reinserido a partir de E.

3. Resultado

Caminho encontrado: B → E → D

Custo total: 2 + 8 = 10

Profundidade: 2 ações

Ordem de retirada: B, E, D

3 nós retirados | **2 nós expandidos**

Apenas **B e E** foram expandidos. **D** não foi expandido.

O resultado depende da ordem de inserção

- Com inserção alfabética **A, C, E**, a DFS segue **E** primeiro e encontra **B → E → D**.
- Neste exemplo, encontra uma solução com 2 ações, mas isso não é uma garantia geral.
- Existe um caminho mais barato: **B → E → C → D**, com custo 8.

A primeira solução encontrada pela DFS não é necessariamente a de menor custo.

Vamos pensar

Aplicar os conceitos a uma nova situação

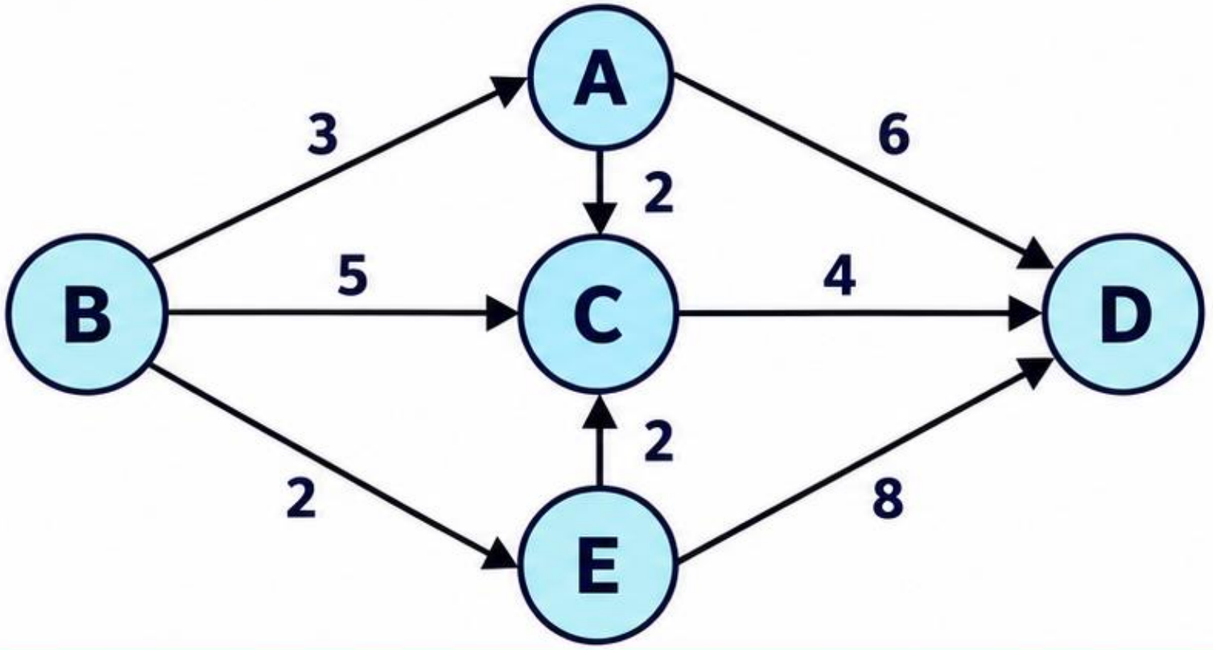
Analise o problema, formule uma resposta e justifique o seu raciocínio.



Alterar a ordem muda a solução?

QUESTÃO

Início: B. Objetivo: D. Os números nos arcos indicam custos.



Alteração: ao expandir B, inserir os sucessores pela ordem **E, C, A** (em vez de A, C, E).

Nos restantes nós, inserir por ordem alfabética.
Marcar cada estado quando entra na fronteira;
não o reinserir nem mudar o seu pai.
Testar o objetivo ao retirar.

Na **BFS:** fila **FIFO.** Na **DFS:** pilha **LIFO,** com o topo à direita.

1 Qual será o próximo nó retirado pela BFS e pela DFS, depois de expandir B?

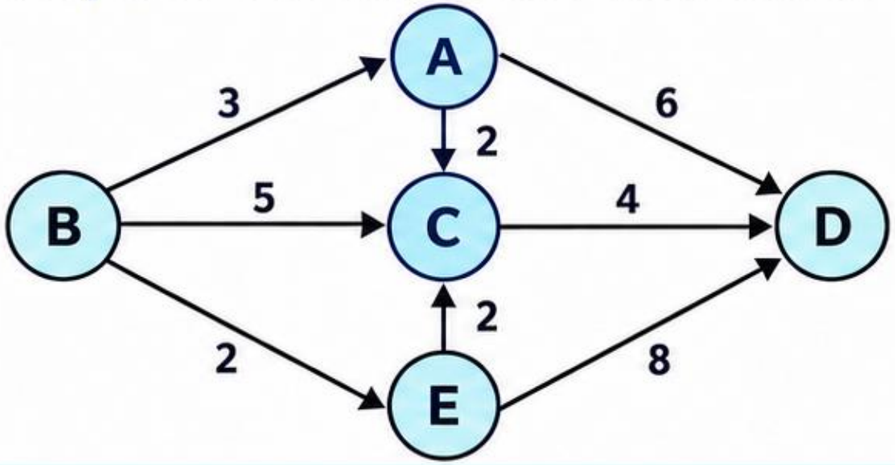
2 Que caminho até D encontra cada algoritmo? Calcule a profundidade e o custo.

3 A mudança de ordem retira à BFS a garantia de encontrar o menor número de ações? Justifique.

Alterar a ordem muda a solução?

QUESTÃO

Início: B. Objetivo: D. Os números nos arcos indicam custos.



Alteração: ao expandir B, inserir os sucessores pela ordem **E, C, A** (em vez de A, C, E).

Nos restantes nós, inserir por ordem alfabética. Marcar cada estado quando entra na fronteira; não o reinserir nem mudar o seu pai. Testar o objetivo ao retirar.

Na **BFS:** fila **FIFO.** Na **DFS:** pilha LIFO, com o topo à direita.

1. Qual será o próximo nó retirado pela BFS e pela DFS, depois de expandir B?
2. Que caminho até D encontra cada algoritmo? Calcule a profundidade e o custo.
3. A mudança de ordem retira à BFS a garantia de encontrar o menor número de ações? Justifique.

SOLUÇÃO

1 Próximo nó

BFS: E — foi o primeiro inserido.

DFS: A — foi o último inserido.

2 Caminhos encontrados

Algoritmo	Caminho	Ações	Custo
BFS	B → E → D	2	2 + 8 = 10
DFS	B → A → D	2	3 + 6 = 9

BFS: [E,C,A] → [C,A,D] → [A,D] → [D] → []
Frente da fila à esquerda. D conserva o pai E.

DFS: [E,C,A] → [E,C,D] → [E,C]
Topo à direita. D tem pai A; termina ao retirar D.

3 Garantia da BFS

Não. A **BFS** continua a explorar por níveis: primeiro os nós a uma ação, depois os nós a duas ações, e assim sucessivamente.
Alterar a ordem de inserção pode mudar o caminho escolhido entre soluções com o mesmo número de ações.
Neste exemplo, B → A → D e B → E → D têm duas ações, o mínimo possível porque não existe B → D direto. Os custos são diferentes: 9 e 10.

A ordem de inserção pode mudar o caminho encontrado; FIFO e LIFO continuam a determinar a exploração.

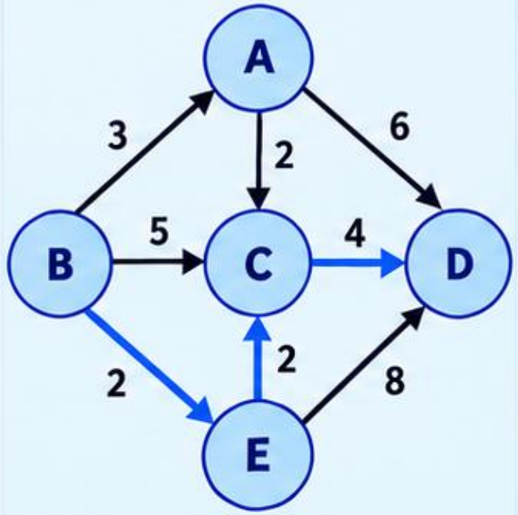
UCS: procura de custo uniforme

Escolher o menor custo acumulado desde o início

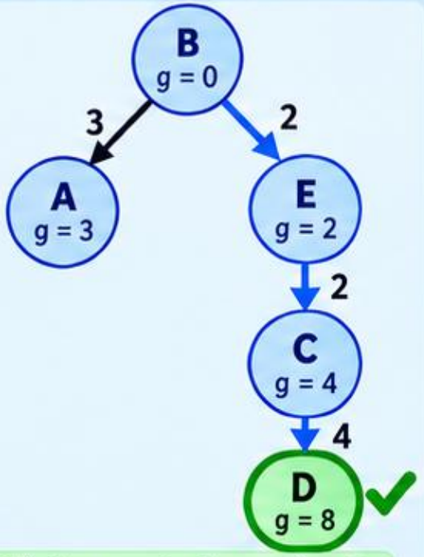
1. Do grafo aos melhores caminhos

Partimos de **B** e procuramos D. O estado é apenas o local.

Grafo completo



Árvore final de melhores caminhos



Objetivo encontrado ao retirar D.

Nas setas: custo da ação. **Nos nós**: custo acumulado *g*.

g(filho) = g(pai) + custo da ação

Caminho final: B → E → C → D. **Custo:** 2 + 2 + 4 = 8.
A árvore mostra os pais finais, após melhorar os caminhos.

2. Uma fila de prioridade

Retiramos o menor *g* de toda a fronteira.

Depois de expandir B:

Inserir ou atualizar por prioridade

Retirar menor g

E	A	C
g = 2	g = 3	g = 5

Representação ordenada por g; **não é uma *fila FIFO*.**

Próximo nó a retirar: E

Melhorar caminhos já encontrados

Ao expandir E, C melhora: 5 → 2 + 2 = 4.
O pai de C passa a ser E.

A	C	D
g = 3	g = 4	g = 10

Agora retiramos A, que já estava na fronteira.

A prioridade depende do custo total do caminho, não apenas da última ação.

3. Algoritmo em linguagem simples

1. Inserir o nó inicial com g = 0 e registar esse custo.
2. Enquanto a fronteira não estiver vazia:
   - Retirar o nó com menor g.
   - Se satisfaz o objetivo, devolver o caminho.
   - Gerar sucessores e calcular os novos custos.
   - Se o estado é novo ou o custo melhora: atualizar custo, pai, ação e prioridade.
3. Se a fronteira se esgotar, devolver falha.

Neste exemplo, substituímos a entrada anterior quando o custo melhora.

Em empate de custo, sai o primeiro inserido.

O que garante?

Num grafo finito, com custos não negativos e atualização correta dos caminhos, encontra uma solução de menor custo, se existir.

Em espaços infinitos, exigem-se condições adicionais: ramificação finita e custo por ação ≥ ε > 0.

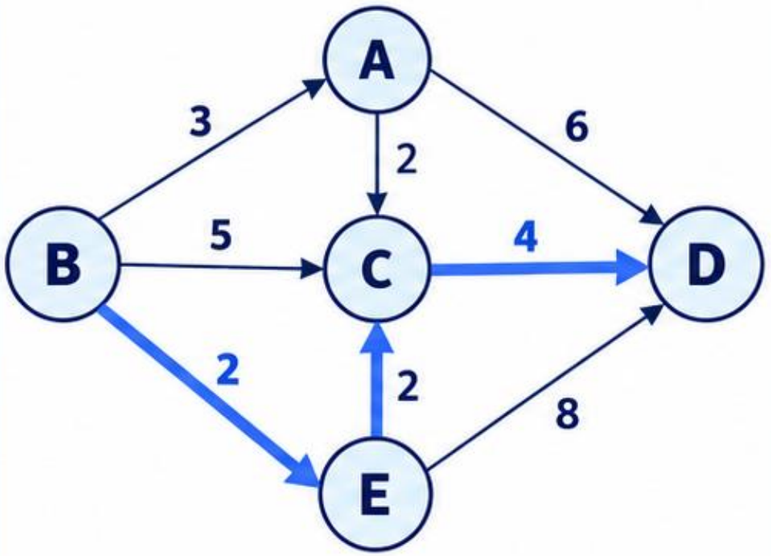
O objetivo é aceite ao retirar o nó de menor custo, não quando é gerado pela primeira vez.

UCS: execução passs o a passo

Atualizar caminhos até retirar o objetivo com o menor custo

Início: B • Objetivo: D • Selecionar menor g • Atualizar se o custo melhorar • Testar ao retirar

1. O mesmo grafo



Estado: apenas o local. *g* é o custo acumulado desde B.

Um caminho mais barato atualiza o custo, o pai e a prioridade do nó.

2. Evolução da fila de prioridade

Menor *g* à esquerda. Escrevemos estado:custo.

Operação	Fronteira	Atualizações
Inicializar	[B:0]	Custo inicial 0.
Retirar e expandir B	[E:2, A:3, C:5]	Inserir os três sucessores.
Retirar e expandir E	[A:3, C:4, D:10]	C melhora de 5 para 4; inserir D com 10.
Retirar e expandir A	[C:4, D:9]	Até C: 3+2=5, não melhora 4. D melhora de 10 para 9.
Retirar e expandir C	[D:8]	D melhora de 9 para 8; pai passa a C.
Retirar D	[]	Objetivo encontrado; não expandir D.

D é gerado com custo 10. A procura continua e melhora esse custo para 9 e depois 8.

A tabela mostra apenas as entradas atuais; as anteriores são substituídas.

3. Resultado

Caminho encontrado:
B → E → C → D

Custo mínimo:
2 + 2 + 4 = 8

Profundidade:
3 ações

Ordem de retirada:
B, E, A, C, D

5 nós retirados

4 nós expandidos

Pais finais:
D tem pai C; C tem pai E; E tem pai B.

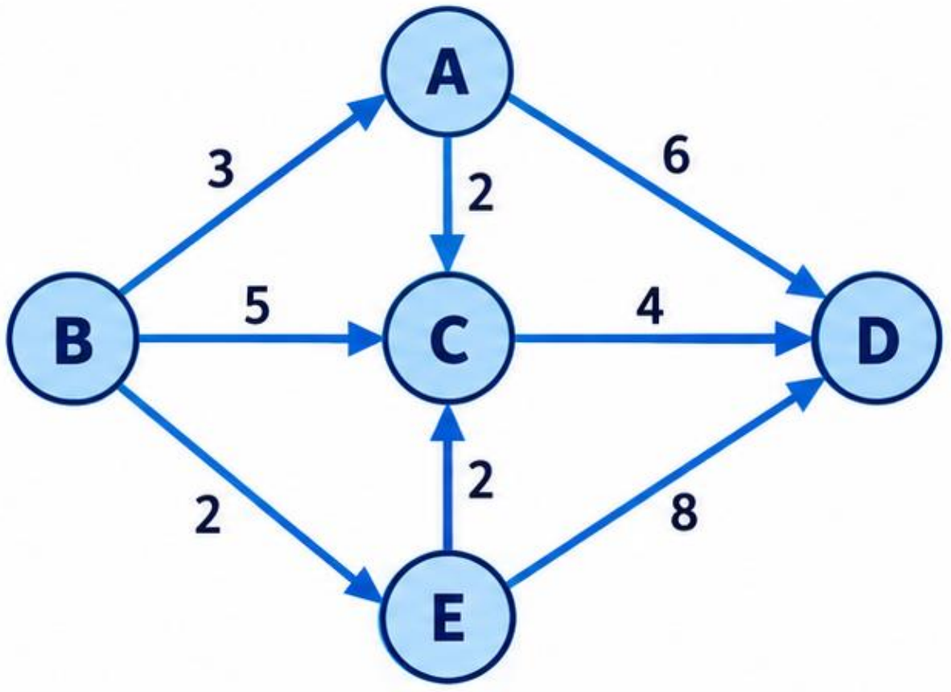
Porque não terminamos ao gerar D?

O primeiro caminho até D custa 10. Só aceitamos o objetivo quando é retirado como o nó de menor custo da fronteira.
Com os custos não negativos deste grafo, o caminho devolvido tem o menor custo.

A UCS encontra custo 8 com 3 ações. A BFS encontra 2 ações com custo 9: os critérios são diferentes.

Comparar BFS, DFS e UCS

Três estratégias, o mesmo grafo e diferentes regras de seleção



Partida: B com 12 unidades de energia

Objetivo: chegar a D com energia ≥ 2

Resultados desta execução

Estratégia	Próximo nó	Caminho	Ações	Custo	Energia final
BFS	mais antigo	B → A → D	2	9	3
DFS	mais recente	B → E → D	2	10	2
UCS	menor custo acumulado	B → E → C → D	3	8	4

Ordem de retirada

Estratégia	Ordem	Retirados	Expandidos
BFS	B, A, C, E, D	5	4
DFS	B, E, D	3	2
UCS	B, E, A, C, D	5	4

Retirado: selecionado da fronteira. **Expandido:** gerou sucessores.
D é retirado, mas não expandido.

Objetivo testado ao retirar; sucessores por ordem alfabética; **BFS** e **DFS** marcam **ao inserir**; **UCS** atualiza a fronteira se encontrar custo menor.
Para controlar repetições usa-se o **local**; a energia final calcula-se pelo custo do caminho.

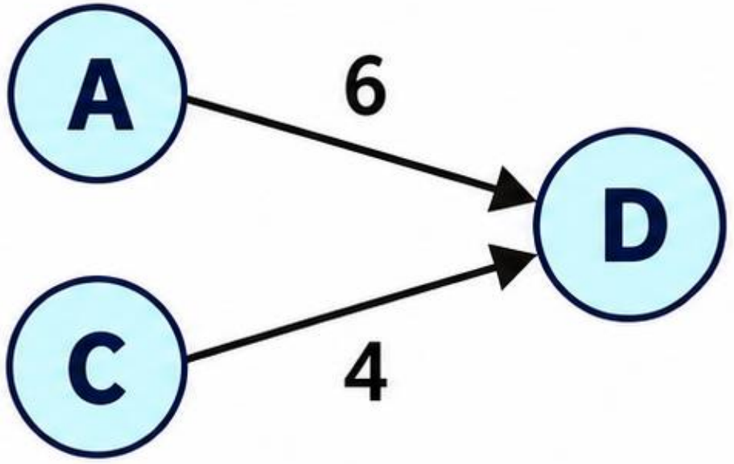
Menos ações não implica menor custo.

Neste exemplo, a UCS encontrou o menor custo; a DFS retirou menos nós.

UCS: encontrar o objetivo basta para terminar?

QUESTÃO

Numa execução da UCS, D é o objetivo.
Os custos dos arcos são não negativos.



OPEN = [A:3, C:4, D:10]

A:3 significa que o melhor caminho conhecido desde o início até A custa 3.

Nas próximas expansões, considerar apenas estas duas ligações. D não tem sucessores.

Ana: «D já foi encontrado. Podemos terminar com custo 10.»

Bruno: «Devemos escolher C, porque C → D é a ligação mais barata.»

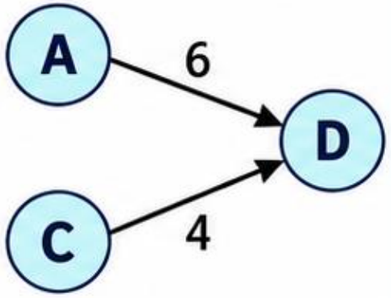
Carla: «Devemos retirar A, embora C possa conduzir a uma solução melhor.»

1. **Quem está correto? Justifique usando a regra de seleção da UCS.**
2. Mostre a evolução de OPEN até confirmar D. Indique as alterações do custo e do pai de D.
3. Explique por que explorar A primeiro não impede encontrar a solução mais barata.

UCS: encontrar o objetivo basta para terminar?

QUESTÃO

Numa execução da UCS, D é o objetivo.
Os custos dos arcos são não negativos.



OPEN = [A:3, C:4, D:10]

A:**3** significa que o melhor caminho conhecido desde o início até A custa 3.

Nas próximas expansões, considerar apenas estas duas ligações.
D não tem sucessores.

Ana: «D já foi encontrado. Podemos terminar com custo 10.»

Bruno: «Devemos escolher C, porque C → D é a ligação mais barata.»

Carla: «Devemos retirar A, embora C possa conduzir a uma solução melhor.»

1. Quem está correto? Justifique usando a regra de seleção da UCS.
2. Mostre a evolução de OPEN até confirmar D. Indique as alterações do custo e do pai de D.
3. Explique por que explorar A primeiro não impede encontrar a solução mais barata.

SOLUÇÃO

1 Escolher pelo custo acumulado

Carla está correta: A:3 tem o menor custo acumulado em OPEN.

Ana confunde gerar o objetivo com confirmar a solução ótima.
Bruno compara apenas o custo da última ligação.

2 Atualizar a fronteira e os pais

Operação	OPEN após o passo	Atualização de D
Expandir **A:3**	[C:4, D:9]	3 + 6 = 9 < 10; pai(D) = A
Expandir **C:4**	[D:**8**]	4 + 4 = 8 < 9; pai(D) = C
Retirar D:**8**	[]	Confirmar objetivo; não expandir.

3 As alternativas continuam disponíveis

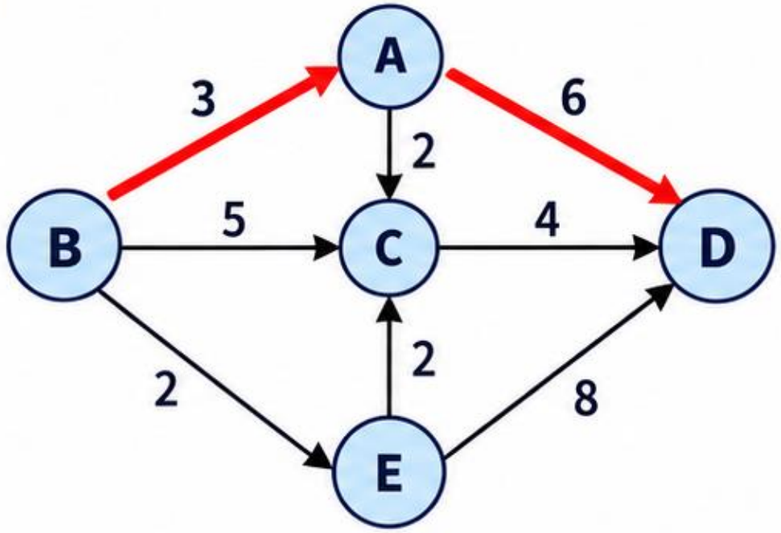
Explorar A não obriga a continuar pelo seu ramo. C permanece em OPEN e é retirado antes de D:9.
Ao explorar C, o custo de D melhora para 8 e o seu pai passa a C.
Reconstrói-se o caminho seguindo os pais; não se usa a ordem de retirada como percurso.

Custo final: 8.
Confirmar D quando é retirado com o menor custo da fronteira.

Verificar uma solução de procura

Uma solução candidata pode ser válida sem ser ótima.

1. Problema



Início: B com 12 unidades de energia

Objetivo: chegar a D com energia ≥ 2

2. A solução é válida?



- ✓ **Ligações permitidas:** sim
- ✓ **Energia suficiente em cada ação:** sim
- ✓ **Chega a D:** sim
- ✓ **Energia final ≥ 2:** sim, termina com 3

B → A → D é válida

Custo: 3 + 6 = 9

3. É ótima em custo?

Caminho	Custo	Energia final
B → A → D	9	3
B → E → C → D	8	4

✗ **B → A → D não é ótimo:** existe uma solução válida com custo inferior.

ⓘ **Para provar que o custo 8 é ótimo,** é necessária uma garantia ou uma verificação adicional.

Para verificar qualquer solução

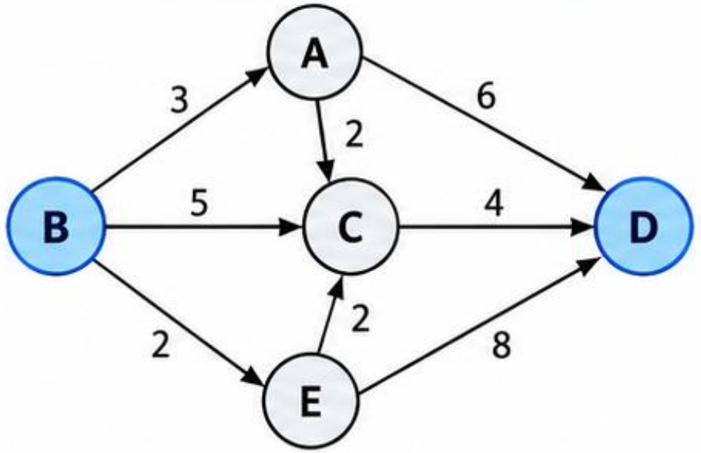
1. Verificar as pré-condições de cada ação.
2. Verificar as restrições durante o percurso e no final.
3. Calcular o custo segundo o critério do problema.
4. Confirmar a garantia ou prova antes de afirmar que é ótima.
5. Confirmar que o modelo representa a tarefa real.

💡 **Uma solução válida pode não ter o menor custo.**

Espaço de estados e alcançabilidade

Que estados podem ser atingidos a partir do estado inicial?

1. O modelo



$S = \{A, B, C, D, E\}$
Estado inicial: B

Modelo simplificado: estado = local; ignoram-se os custos e as restrições de energia.

Espaço de estados S: todos os estados considerados no modelo.
Conjunto alcançável R: estados atingíveis a partir do estado inicial.

2. Construir o conjunto alcançável

Passo	Novos estados	Conjunto acumulado
0	B	$R_0 = \{B\}$
até 1 ação	A, C, E	$R_1 = \{B, A, C, E\}$
até 2 ações	D	$R_2 = \{B, A, C, E, D\}$
passo seguinte	nenhum	$R_3 = R_2$

R_k contém os estados alcançáveis em até k ações; por isso mantém os estados anteriores.

$$R_{k+1} = R_k \cup \text{sucessores}(R_k)$$

Quando não surgem novos estados, atingimos um ponto fixo.

3. Conclusões e algoritmo

```
R ← {B}
Repetir:
    adicionar a R os sucessores dos estados em R
Parar quando não surgirem novos estados
Devolver R
```

- Neste exemplo, $R = S = \{A, B, C, D, E\}$.
- $D \in R$: existe um caminho de B até D no modelo simplificado.
- Se nenhum objetivo pertencer a R, não existe solução nesse modelo.
- Num grafo finito, o processo termina; num espaço infinito, pode não terminar.

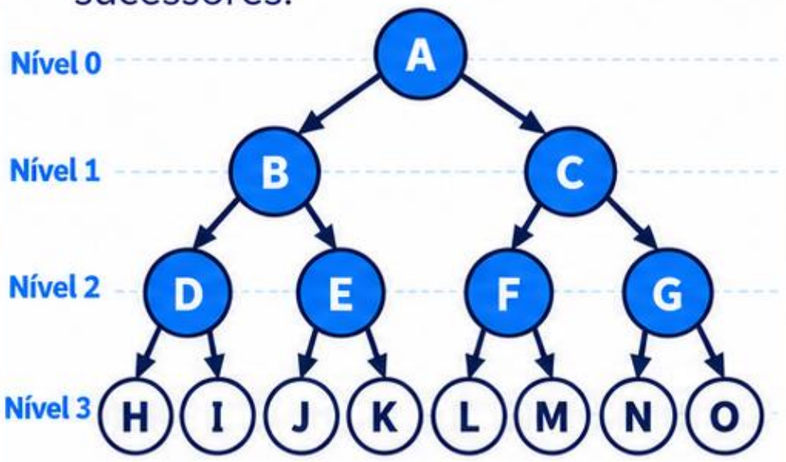
O espaço de estados contém os estados do modelo; a alcançabilidade identifica os que são atingíveis a partir do início.

Tempo da procura

Que trabalho realiza um algoritmo de procura?

1. O que acontece aos nós?

Gerado: o nó é criado.
Examinado: o estado é analisado, por exemplo para testar o objetivo.
Expandido: são gerados os seus sucessores.



15 nós gerados, incluindo A.
7 nós expandidos: A, B, C, D, E, F e G.
8 nós aguardam exploração: H a O.
O desenho não indica quantos testes do objetivo foram feitos.

2. Como estimar o tempo?

Tempo total
≈ número de operações
× tempo médio por operação

Exemplos de operações:

- Criar um nó.
- Testar se é o objetivo.
- Inserir ou retirar um nó da fronteira.

Exemplo ilustrativo

100 operações × 2 ms = 200 ms

Assumindo uma média de 2 ms por operação.

As operações podem ter custos diferentes.

3. Como cresce o número de nós?

Exemplo: cada nó gera 2 sucessores.
O nó inicial está no nível 0.

Nível	Nós nesse nível	Total até aqui
0	1	1
1	2	3
2	4	7
3	**8**	**15**

Num nível: multiplicamos por 2.
No nível 3: $2^3 = 8$ nós.

No total: somamos os níveis.
Até ao nível 3: $1 + 2 + 4 + 8 = 15$ nós.

Regra geral
b = sucessores por nó; ℓ = nível
Num nível: $n_\ell = b^\ell$
Até esse nível: $N = 1 + b + \cdots + b^\ell$

Se cada nó antes do nível ℓ gerar exatamente b sucessores.

Mais nós podem exigir mais operações.
A procura pode parar antes, ao encontrar o objetivo.

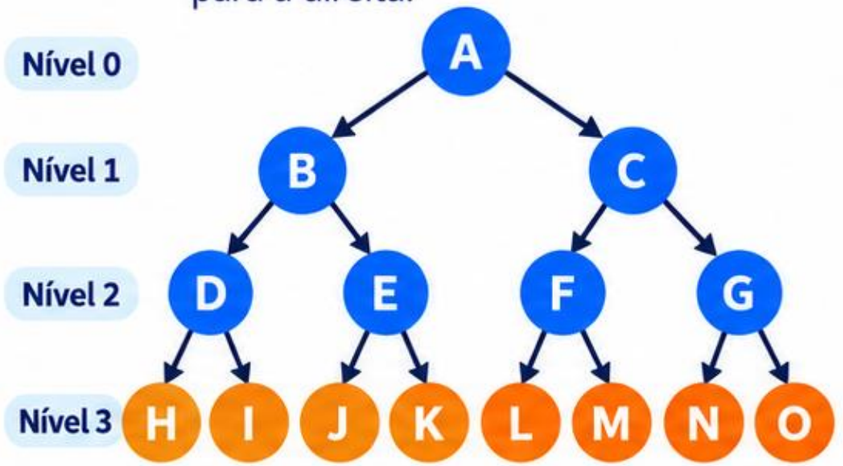
O tempo depende da quantidade de operações e do custo de cada uma.

Memória da procura

Quanta informação fica guardada ao mesmo tempo?

1. Que nós estão em espera?

Exemplo: exploração por níveis, da esquerda para a direita.



Azul: nós já expandidos.
Laranja: nós gerados que aguardam exploração.

Fronteira após expandir G

H	I	J	K	L	M	N	O

Neste momento: 8 nós em espera.

A árvore mostra os nós gerados; a fronteira contém os que aguardam exploração.

2. Quanto espaço ocupam?

Espaço dos registos da fronteira ≈ número de nós em espera × espaço médio por registo

Um registo guarda informação de um nó.

Exemplo ilustrativo

8 registos × 80 bytes = 640 bytes

Assumindo 80 bytes por registo.

640 bytes referem-se apenas a estes 8 registos.

A memória total inclui também a estrutura da fronteira e outros dados necessários, como ligações aos nós anteriores.

Em grafos, podem existir ainda registos para controlar estados repetidos.

3. Qual é o máximo?

Contamos o maior número de nós em espera durante a procura.

Evolução da fronteira neste exemplo

Momento	Nós em espera
Início: apenas A	1
Após expandir A	2
Após expandir B	3
Após expandir C	4
Após expandir D	5
Após expandir E	6
Após expandir F	7
Após expandir G	**8**

K = max(1, 2, 3, 4, 5, 6, 7, 8) = 8

K é o máximo de nós na fronteira.

Não somamos as ocupações de momentos diferentes.

Se continuarmos a retirar as folhas H a O, a fronteira diminui.

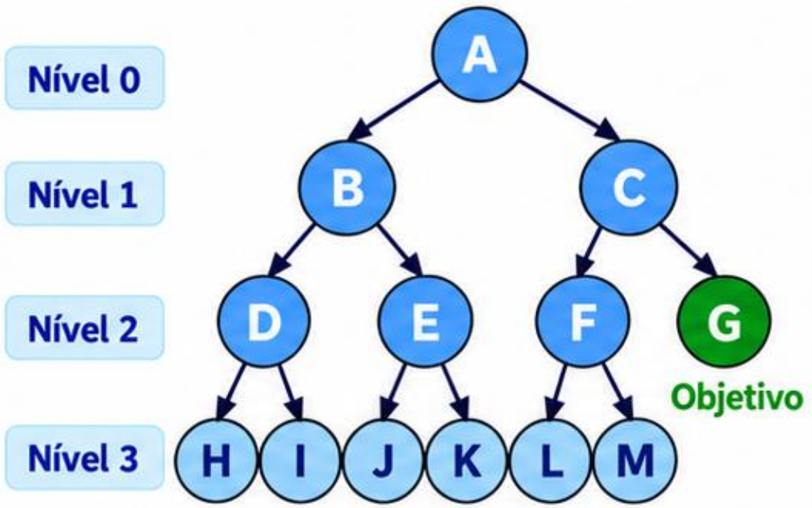
O pico da fronteira é 8 nós. A memória total inclui também outros registos.

BFS: tempo e memória

Procura em largura: explorar um nível antes do seguinte

1. Como funciona?

Fila: o primeiro a entrar é o primeiro a sair.



Ordem de retirada até G:

A, B, C, D, E, F, G

Inserir os filhos da esquerda para a direita, no fim da fila.

Testar o objetivo ao retirar; G não é expandido.

2. Tempo: como cresce?

b: máximo de filhos por nó.
d: nível do objetivo mais próximo.
m: profundidade máxima da árvore.

Neste exemplo: $b = 2$ e $d = 2$.

Antes de retirar G, a BFS expande D, E e F e gera filhos no nível 3.

Nível adicional: $d + 1 = 3$

Limite geral do tempo:

$$O(b^{d+1})$$

O que nos diz a fórmula?

$O(\cdot)$ descreve o crescimento no pior caso.
Não dá a contagem exata deste exemplo.

Por níveis: mais filhos e maior profundidade podem aumentar muito o trabalho.

3. Memória: o que fica em espera?

Alguns momentos da execução

Após expandir	Fronteira	Nós
A	B, C	2
B	C, D, E	3
C	D, E, F, **G**	4
D	E, F, **G**, **H, I**	5
E	F, **G**, **H, I, J, K**	6
F	**G, H, I, J, K, L, M**	7

Próximo a sair: à esquerda.

Contagens reais deste exemplo

Nós gerados, incluindo A:	**13**
Nós retirados, incluindo G:	**7**
Nós expandidos, sem G:	**6**
Máximo na fronteira:	**7**

Limite geral da fronteira:

$$O(b^{d+1})$$

Neste exemplo: até 7 nós em espera.

Procura em árvore; $b \geq 2$ e m finito. Objetivo testado ao retirar.
Custo por nó e tamanho dos registos constantes.

A fronteira é parte da memória;
outros registos também ocupam espaço.

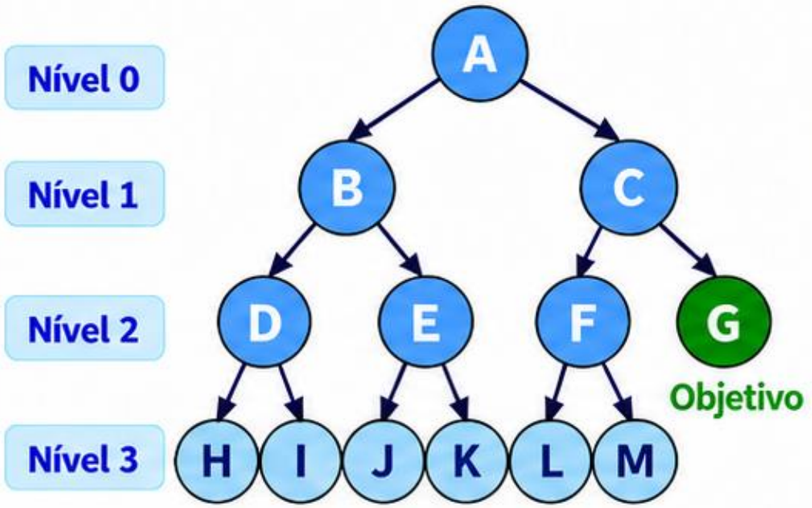
A fórmula explica o crescimento. A execução dá as contagens reais.

DFS: tempo e memória

Procura em profundidade: aprofundar um ramo antes de tentar outro

1. Como funciona?

Pilha: o último a entrar é o primeiro a sair.



Ordem de retirada até G:

A, B, D, H, I, E, J, K, C, F, L, M, G

Inserir primeiro o filho da direita na pilha, para retirar primeiro o da esquerda.

Testar o objetivo ao retirar; G não é expandido.

2. Tempo: como cresce?

b: máximo de filhos por nó.
d: nível do objetivo mais próximo.
m: profundidade máxima da árvore.

Neste exemplo: $b = 2$ e $m = 3$.

G está no nível 2, mas a DFS explora primeiro outros ramos até ao nível 3.

Profundidade máxima: $m = 3$

Limite geral do tempo:

$$O(b^m)$$

O que nos diz a fórmula?

$O(\cdot)$ descreve o crescimento no pior caso.
Não dá a contagem exata deste exemplo.

Por ramos: a DFS pode explorar quase toda a árvore antes de encontrar G.

3. Memória: o que fica em espera?

Alguns momentos da execução

Após expandir	Fronteira	Nós
A	B, C	2
B	D, E, C	3
D	H, I, E, C	4
E	J, K, C	3
F	L, M, G	3
M	G	1

Próximo a sair: à esquerda.

Contagens reais deste exemplo

Nós gerados, incluindo A:	**13**
Nós retirados, incluindo G:	**13**
Nós expandidos, sem G:	**12**
Máximo na fronteira:	**4**

Limite geral da fronteira:

$$O(b \times m)$$

Neste exemplo: até 4 nós em espera.

Procura em árvore; $b \geq 2$ e m finito. Objetivo testado ao retirar.
Custo por nó e tamanho dos registos constantes.

A fronteira é parte da memória;
outros registos também ocupam espaço.

A fórmula explica o crescimento. A execução dá as contagens reais.

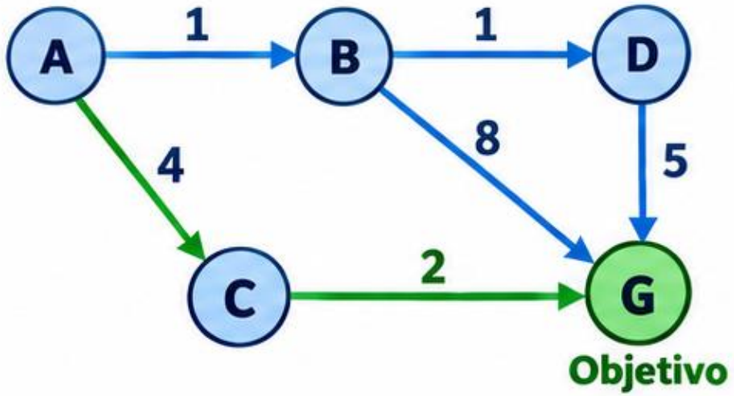
UCS: tempo e memória

Procura de custo uniforme: explorar primeiro o caminho com menor custo acumulado.

1. Como funciona?

Usa **uma fila de prioridade**, ordenada pelo **custo acumulado $g(n)$**.

$g(n)$ = soma dos custos desde o início até n.



Solução: A → C → G

2 ações, custo total 4 + 2 = **6**.

A → B → D → G custa 1 + 1 + 5 = **7**.

O objetivo é aceite quando sai da fila, não quando é gerado.

2. Porque aumenta o tempo?

Pode explorar muitos caminhos baratos antes de retirar o objetivo.

Fila ordenada por custo

Após expandir	Fronteira
A	B(1), C(4)
B	D(2), C(4), G(9)
D	C(4), G(7)
C	G(6)

Ao encontrar um caminho mais barato para G, atualizamos a sua prioridade.

Depois, retiramos G com custo 6 e paramos.

Neste exemplo, mantemos a melhor entrada por estado.

Custos pequenos podem permitir muitos passos antes de atingir o custo da solução.

3. Tempo e memória

b = máximo de sucessores por nó

C^* = custo da solução ótima

$\varepsilon > 0$ = limite inferior do custo de cada ação

Limite clássico para nós gerados e memória:

$$O\left(b^{1+\left\lfloor \frac{C^*}{\varepsilon} \right\rfloor}\right)$$

$\lfloor C^*/\varepsilon \rfloor$ limita o número de passos de um caminho com custo até C^*.

O **+1** admite sucessores gerados antes de retirar o objetivo.

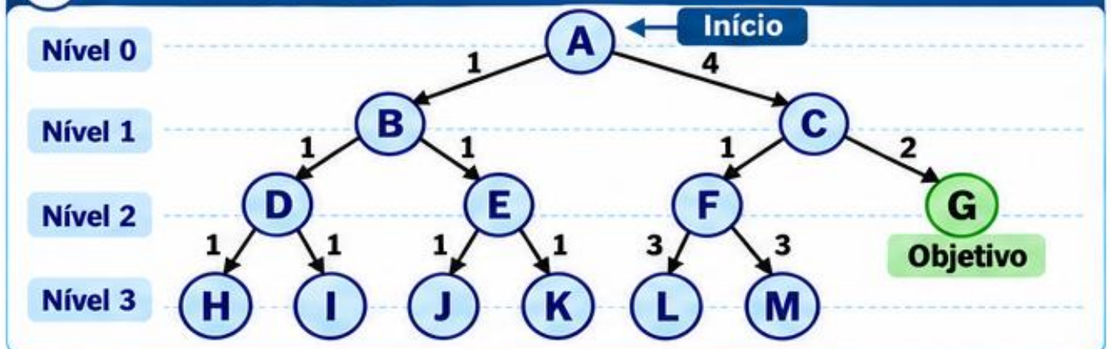
A **fronteira** guarda alternativas por explorar; em grafos, há também registos dos estados alcançados.

O tempo real inclui o custo de gerir a fila de prioridade.

Fórmula para o pior caso na procura em árvore, com $b \geq 2$ e custos de ação $\geq \varepsilon > 0$.
Conta nós; com um heap, cada inserção ou remoção pode custar $O(\log F)$, sendo F o tamanho da fila.

A UCS procura o menor custo total. Muitos caminhos baratos podem exigir tempo e memória.

1 A mesma árvore



2 Como contamos?

Objetivo testado ao retirar.

BFS e DFS: explorar primeiro à esquerda; na DFS, inserir primeiro à direita.

UCS: menor custo acumulado; empates por ordem alfabética.

Tempo: trabalho realizado.

Memória da fronteira: máximo de nós em espera.

3 Resultados nesta árvore

Medida	BFS · fila	DFS · pilha	UCS · prioridade
Nós retirados, incluindo G	7	13	11
Nós expandidos, sem G	6	12	10
Máximo de nós em espera	7	4	5

4 Conclusões sobre tempo e memória

BFS — explora por níveis

TEMPO: encontra soluções pouco profundas sem seguir ramos longos. Se a solução estiver muito longe, o trabalho cresce muito.

MEMÓRIA: pode guardar muitos nós dos níveis seguintes ao mesmo tempo.

Neste exemplo: menos nós retirados (7); até 7 em espera.

DFS — aprofunda um ramo

TEMPO: pode gastar muito tempo num ramo errado, antes de tentar outro.

MEMÓRIA: na procura em árvore, costuma guardar menos nós: o ramo atual e as opções por tentar.

Neste exemplo: mais nós retirados (13); até 4 em espera.

UCS — menor custo primeiro

TEMPO: pode explorar muitos caminhos baratos antes de encontrar a solução de menor custo. Também gere prioridades.

MEMÓRIA: pode acumular muitos caminhos na fila com prioridade.

Neste exemplo: 11 nós retirados; até 5 em espera.

5 Limites de pior caso na procura em árvore

Estratégia	Tempo	Memória da fronteira	O que significa?
BFS	$O(b^{d+1})$	$O(b^{d+1})$	**Tempo:** depende de b e da profundidade do objetivo mais próximo (d). **Memória:** guarda muitos nós dos níveis que vai explorando.
DFS	$O(b^{m})$	$O(b \times m)$	**Tempo:** depende de b e da profundidade máxima da árvore (m), mesmo que exista um objetivo pouco profundo. **Memória:** guarda o ramo atual e as alternativas por explorar: $O(b \times m)$.

Parâmetros desta árvore: b = máximo de filhos por nó = 2; d = profundidade do objetivo mais próximo = 2; m = profundidade máxima = 3.

$O(\cdot)$ indica crescimento, não a contagem exata. $b \geq 2$; m finito; custo por nó constante; objetivo testado ao retirar.

Nesta árvore, $d + 1 = m = 3$: os limites temporais coincidem, mas BFS retira 7 nós e DFS retira 13.

Nesta árvore: BFS faz menos trabalho de exploração; DFS mantém menos nós em espera.

Contar nós não mede segundos. Estes resultados não são uma classificação universal. | A fronteira é parte da memória. Em grafos, guardar os estados já alcançados também ocupa espaço.

6 Distinção essencial

Menos ações, menor custo ou apenas uma solução?

1 O que procura cada estratégia?



BFS — por níveis

Encontra uma solução com o menor número de ações.



DFS — em profundidade

Explora um ramo de cada vez. Não garante menos ações nem menor custo.



UCS — por custo acumulado

Encontra uma solução de menor custo, nas condições indicados abaixo.

2 Exemplo ilustrativo: início B, objetivo D

Estratégia	Caminho apresentado	Ações	Custo total
BFS	B → A → D	2	9
DFS	B → E → D	2	10
UCS	B → E → C → D	3	8

O caminho pode depender da ordem dos sucessores e dos desempates.



Mais ações podem custar menos:
3 ações custam 8; 2 ações custam 9.

Este exemplo é diferente da árvore com início A e objetivo G.

3 Garantias num espaço finito

Com controlo correto de estados repetidos; existe uma solução.

Estratégia	Encontra solução?	Menos ações?	Menor custo?	O que significa?
BFS	Sim	Sim	Garantido se os custos por ação forem iguais e positivos.	Escolher quando importa reduzir o número de ações.
DFS	Sim	Sem garantia	Sem garantia	Pode encontrar uma solução, mas esta pode ser mais longa ou mais cara.
UCS	Sim	Sem garantia	Sim, com custos não negativos e atualização correta.	Escolher quando importa minimizar a soma dos custos.

- Sem garantia não significa impossível: pode acontecer num caso concreto.
- Na UCS, atualizar o melhor custo de cada estado e testar o objetivo ao retirar da fila de prioridade.

4 E em espaços infinitos?

- **BFS:** pode ficar presa num ramo infinito.
- **BFS:** completa com ramificação finita e solução a profundidade finita.
- **UCS:** completa com ramificação finita, solução de custo finito e custo por ação $\geq \varepsilon > 0$.
- ε é um valor mínimo positivo para o custo de qualquer ação.

5 Três conceitos diferentes

- **Completude:** encontra uma solução, se existir, nas condições indicadas.
- **Menos ações:** minimiza o número de passos do caminho.
- **Menor custo:** minimiza a soma dos custos das ações.

6 Distinção essencial

- **BFS** escolhe pela profundidade; **DFS** segue um ramo; **UCS** escolhe pelo custo acumulado.
- Menos ações não significa menor custo — nem menos trabalho de procura.
- Número de ações = passos da solução. Trabalho de procura = nós explorados até a encontrar.

Vamos pensar

Aplicar os conceitos a uma nova situação

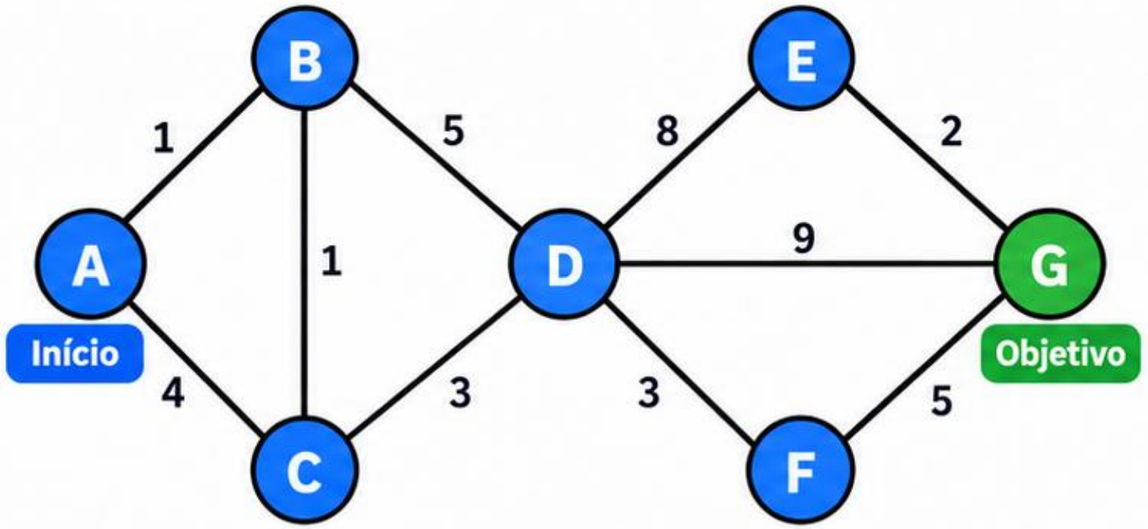
Analise o problema, formule uma resposta e justifique o seu raciocínio.



BFS, DFS e UCS: que caminhos são possíveis?

Exercício — do estado A ao objetivo G

1. Observe o grafo



As ligações podem ser percorridas nos dois sentidos. Os números indicam custos.

2. Analise os três caminhos

P1: A → B → D → G
P2: A → C → D → G
P3: A → B → C → D → F → G

3. Complete e justifique

Assinale todos os caminhos que cada algoritmo pode devolver, variando os desempates.

Algoritmo	P1	P2	P3
BFS · fila	☐	☐	☐
DFS · pilha	☐	☐	☐
UCS · prioridade	☐	☐	☐

BFS — fila (FIFO)

Sai primeiro o nó que entrou primeiro.

DFS — pilha (LIFO)

Sai primeiro o último nó que entrou.

UCS — fila com prioridade

Sai primeiro o nó com menor custo acumulado desde A.

Regras: Procura em grafo: evitar ciclos. Na DFS, seguir sucessores ainda não visitados. Na UCS, testar o objetivo ao retirar. Os desempates podem variar; não há ordem alfabética obrigatória.

Para cada caminho, calcule o número de ligações e o custo total.

Adaptado de Ghent University - IA 2024/2025 - Exercício 6(a)

Vamos pensar

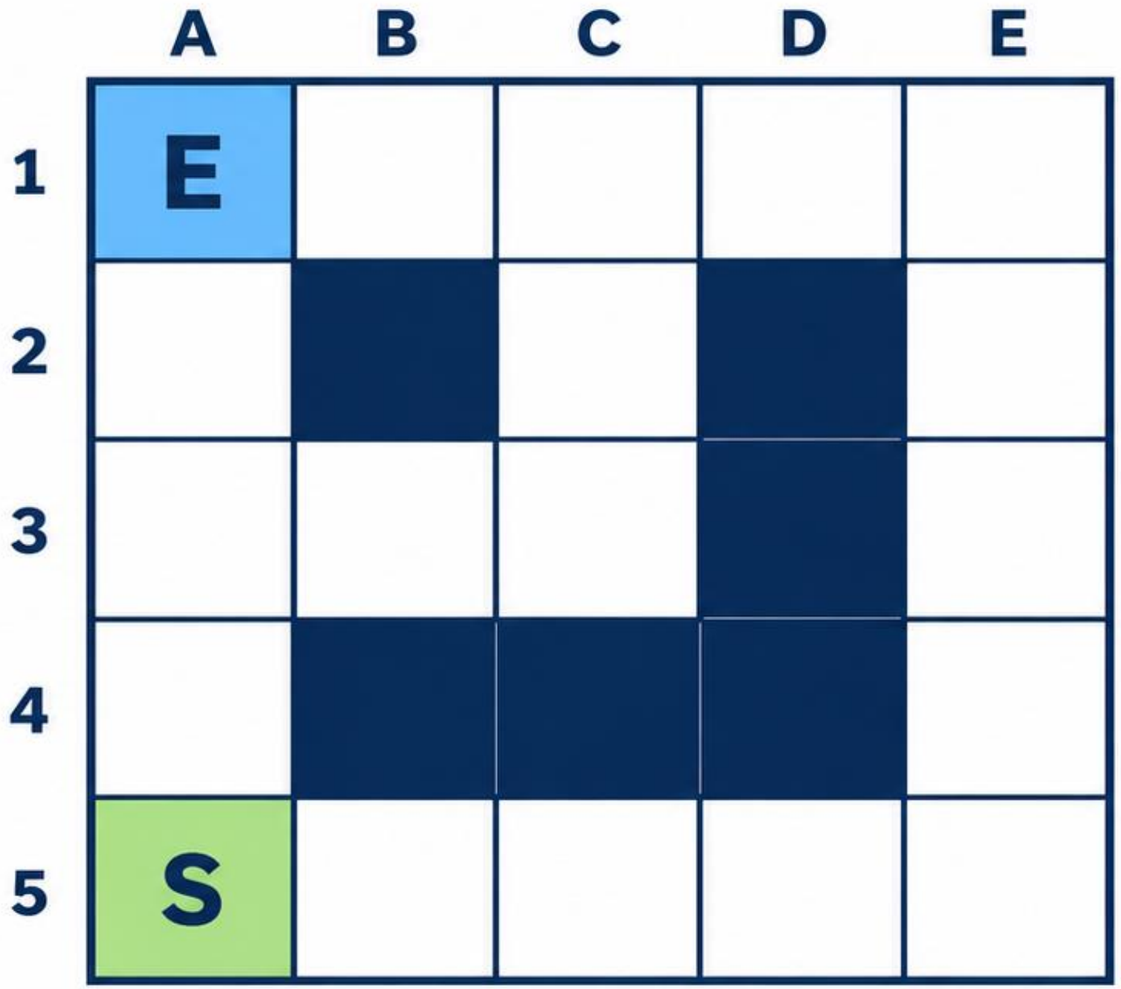
Aplicar os conceitos a uma nova situação

Analise o problema, formule uma resposta e justifique o seu raciocínio.



Labirinto: encontrar a saída com BFS e DFS

Exercício — o mesmo labirinto, duas estratégias



E : entrada | **S** : saída | **Escuro: parede**

Regras

- Movimentos: cima, direita, baixo, esquerda.
- Cada movimento custa 1; não há diagonais.
- Uma posição é marcada quando entra na fronteira e não volta a entrar.
- A saída é testada quando a posição sai da fronteira.
- **BFS:** fila. **DFS:** pilha.
- Na **DFS**, inserir os vizinhos pela ordem inversa para explorar primeiro cima, direita, baixo, esquerda.

O que registar?

1. Ordem das posições retiradas.
2. Caminho encontrado e número de movimentos.
3. Posições retiradas e expandidas.
4. Máximo de posições na fronteira.

Uma posição expandida tem os vizinhos analisados.
A saída é retirada, mas não expandida.

Compare o trabalho realizado e a memória da fronteira, além do caminho encontrado.

Vamos pensar

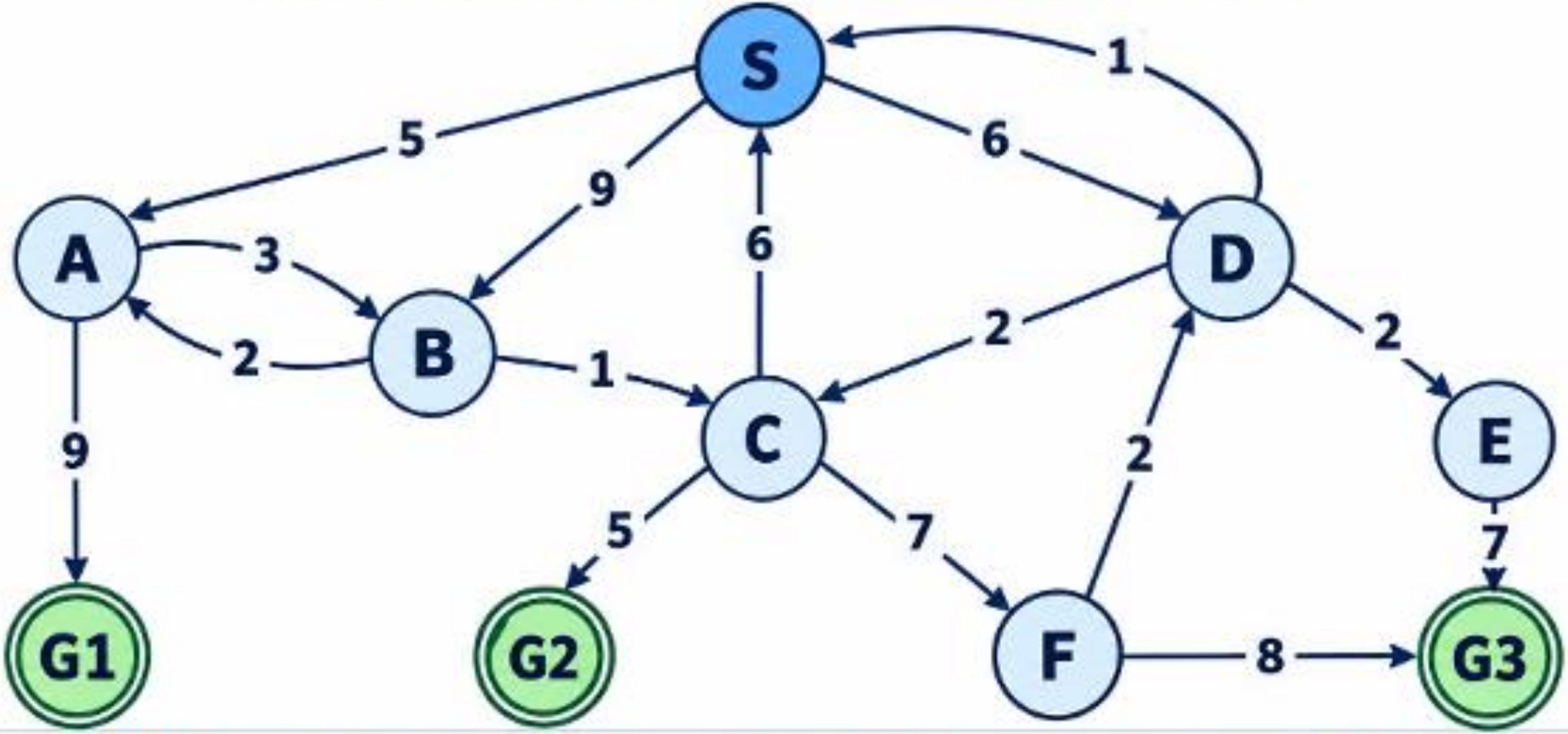
Aplicar os conceitos a uma nova situação

Analise o problema, formule uma resposta e justifique o seu raciocínio.



Grafo de pesquisa — BFS, DFS e UCS

Estado inicial: S | Estados objetivo: G1, G2 e G3



Azul: estado inicial | **Verde:** estados objetivo | **Número no arco:** custo da ação

As setas indicam o sentido permitido. O custo de um caminho é a soma dos custos dos seus arcos.

Vamos pensar

Aplicar os conceitos a uma nova situação

Analise o problema, formule uma resposta e justifique o seu raciocínio.



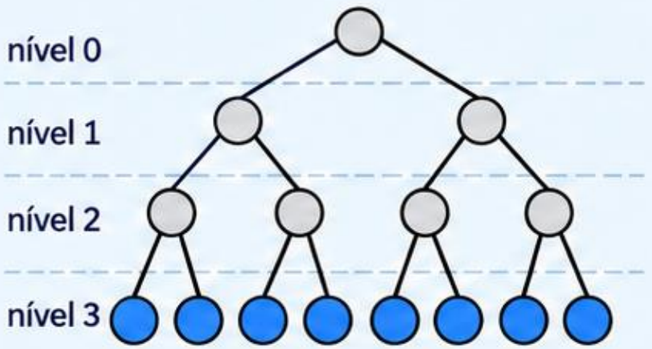
A BFS fica sem memória

Exercício — que alternativa escolher?

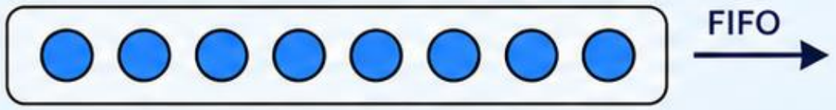
1. O problema

- Todas as ações têm custo 1.
- Sabemos que existe uma solução com exatamente k passos.
- Ainda não sabemos qual é esse caminho.
- Cada estado tem um número finito de sucessores.

Exemplo: k = 4 → procurar uma solução com 4 ações.



Fila da BFS: muitos nós em espera



Esquema ilustrativo; a solução não está representada.

2. A memória esgotou-se

A fila da BFS tornou-se demasiado grande.

Queremos encontrar uma solução usando menos memória.

3. Escolha a melhor alternativa

- ☐ **A. Explorar um ramo de cada vez, sem impor um número máximo de passos.**
  (DFS sem limite de profundidade)
- ☐ **B. Explorar um ramo de cada vez, sem avançar para além de k passos; se k = 4, parar de aprofundar no nível 4.**
  (DFS limitada à profundidade k)
- ☐ **C. Recomeçar a procura com limites cada vez maiores: primeiro 0 passos, depois 1, depois 2, e assim sucessivamente.**
  (Aprofundamento iterativo)
- ☐ **D. Escolher sempre o caminho em espera com menor custo acumulado desde o início.**
  (UCS — procura de custo uniforme)

Justifique a escolha:

1. Que estrutura guarda os nós em espera?
2. Como é usado o valor conhecido de k?
3. Porque são menos adequadas as outras opções?

Compare a fila da BFS com a pilha da DFS e a fila com prioridade da UCS.

Adaptado de UC Berkeley · CS188 · Summer 2024 · Exam Prep 2B · Q2(a)(ii)